

Basics 9 – Ellipsis

Student Information

Integrity Policy: All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies: Yes No

Name:

Date:

Submission Details

Final **Changelist** number:

Verified build: Yes No

Number Tests Passed:

Required Configurations:

Discussion (What did you learn):

Verify Builds

- Follow the Piazza procedure on submission
 - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
 - No – Generated files
 - *.pdb, *.suo, *.sdf, *.user, *.obj, *.exe, *.log, *.pdb, *.db, *.user
 - Anything that is generated by the compiler should not be included
 - No – Generated directories
 - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
 - *.sln, *.cpp, *.h
 - *.vcxproj, *.vcxproj.filters, CleanMe.bat

Standard Rules

Submit multiple times to Perforce

- Submit your work as you go to perforce several times (at least 5)
 - As soon as you get something working, submit to perforce
 - Have reasonable check-in comments
 - Points will be deducted if minimum is not reached

Write all programs in cross-platform C++

- Optimize for execution speed and robustness
- Working code doesn't mean full credit

Submission Report

- Fill out the submission Report
 - No report, no grade

Code and project needs to compile and run

- Make sure that your program compiles and runs
 - Warning level ALL ...
 - NO Warnings or ERRORS
 - Your code should be squeaky clean.
 - Code needs to work "as-is".
 - No modifications to files or deleting files necessary to compile or run.
 - All your code must compile from perforce with no modifications.
 - Otherwise it's a 0, no exceptions

Project needs to run to completion

- If it crashes for any reason...
 - It will not be graded and you get a 0

No Containers

- NO STL allowed {Vector, Lists, Sets, etc...}
 - No automatic containers or arrays
 - You need to do this the old fashion way - **YOU EARNED IT**

Leave Project Settings

- Do NOT change the project or warning level
 - Any changing of level or suppression of warnings is an integrity issue

Simple C++

- No modern C++
 - No Lambdas, Autos, templates, etc...
 - No Boost
- NO Streams
 - Used fopen, fread, fwrite...
- No code in MACROS
 - Code needs to be in cpp files to see and debug it easy
- **Exception:**
 - implicit problem needs templates

Leaking Memory

- If the program leaks memory
 - There is a deduction of 20% of grade
- If a class creates an object using new/malloc
 - It is responsible for its deletion
- Any **MEMORY** dynamically allocated that isn't freed up is **LEAKING**
 - Leaking is **HORRIBLE**, so you lose points

No Debug code or files disabled

- Make sure the program is returned to the original state
 - If you added debug code, please return to original state
- If you disabled file, you need to re-enable the files
 - All files must be active to get credit.
 - Better to lose points for unit tests than to disable and lose all points

No Adding files to this project

- This project will work "as-is" do not add files...
- Grading system will overwrite project settings and will ignore any student's added files and will returned program to the original state

UnitTestFixture file (if provided) needs to be set by user

- Grading will be on the UnitTestFixture settings
 - Please explicitly set which tests you want graded... no regrading if set incorrectly

Due Dates

- See Piazza for due date and time
- Submit program performance in your student directory assignment supplied.
- Fill out your this **Submission Report** and commit to performance
 - o **ONLY** use Adobe Reader to fill out form, all others will be rejected.
 - o Fill out the form and discussion for full credit.

Goals

- Ellipsis in C++
 - o Understand Ellipsis functions
 - o Understand Parameter parsing

Assignments

- General:
 - o Have fun learning Ellipsis:
 - Write two programs and pass the tests
 - `SecondMax()`
 - `parse()`
 - There are only 2 tests
 - So the best score is 2/2
- Program 1: Write an ellipsis program `SecondMax()`
 - o Description:
 - `int SecondMax(int count, ...)`
 - Where the function takes an arbitrary number of parameters
 - The first number is the count, followed by the data
 - Return the 2nd largest number
 - o Example:
 - `val = SecondMax(5, 6, 3, 8, 9, 7);`
 - There are 5 data values, so the 1st parameter is 5
 - returns 8
 - `val = SecondMax(10, 1, 2, 3, 4, 5, 2, 5, 4, 3, 2);`
 - There are 10 data values, so the 1st parameter is 5
 - returns 4
 - o it's the 2nd largest number, since 5 is repeated

- Program 2: Create a standalone program called *parse* that reads into custom chunk and name run-time file format. *Pain And Rare Suffering Exercise*

- o Standalone executable

- ChunkType – only used for this programming assignment

```
enum ChunkType
{
    VERTS_TYPE,
    NORMS_TYPE,
    ANIM_TYPE,
    TEXTURE_TYPE,
    UV_TYPE
};
```

- Reads and parses the arguments

- Return 0
 - o Successful - no parsing errors detected
 - Return -1
 - o Detects a parsing error

- o Formats

- Parameter formats:
 - *parse* < options >
 - options parameters (order isn't important)
 - [-t CHUNK_TYPE]
 - o set chunk type
 - [-n CHUNK_NAME]
 - o set chunk name (max 19 characters, 20 including the null)

- o Example of the full command line

- Successful (returns 0)
 - *parse -t UV_TYPE -n player_1*
 - *parse -n player_1 -t UV_TYPE*
 - *parse -T NORMS_TYPE -N player_1*
 - *parse -t VERTS_TYPE -n Fiat*
 - Unsuccessful – causes error (returns -1)
 - *parse -R UV_TYPE -n player_1*
 - *parse -n player_8888888888881 -t UV_TYPE*
 - *parse -t NORM_TYPE -n player_1*
 - *parse -t VERTS_TYPE -n <missing>*

Validation

Simple checklist to make sure that everything is submitted correctly

- Is the project compiling and running without any errors or warnings?
- Does the project run **ALL** the unit tests execute without crashing?
- Is the submission report filled in and submitted to performe?
- Follow the verification process for performe
 - Is all the code there and compiles “as-is”?
 - No extra files
- Is the project leaking memory?

Hints

Most assignments will have hints in a section like this.

- This is pretty easy Basic assignment
 - Learn Ellipsis by look on the internet
- I expect this assignment to be completed quickly for most of the students
 - Please make sure you fully understand this code without a debugger.
 - Many little lessons here for those who put in the effort.
- Using the debugger parameters to test and debug the parsing
 - Look it up
- Enjoy